12_vit_100m_3d

October 1, 2026

1 3D ViT on 100 m tiles

Each 100 m tile is a 10×10 grid of AlphaEarth cells. Empty slots are masked. Each occupied cell is 9 PLS-DA scores (the smallest k within 0.02 LORO F1 of the 64-D logistic). A transformer (width 128) mixes those cells. The class token is then mapped to **3 dimensions**. That 3D vector is the encoder output.

Loss on the 3D vector:

$$\mathcal{L} = \mathrm{BCE} + \lambda\,\mathcal{L}_{\mathrm{SupCon}}, \quad \lambda = 1,\ \tau = 0.1$$

Same-class tiles are pulled together. RES and NRES are pushed apart. Each metro is its own 3D coordinate system. The scatters below plot those three encoder outputs.

Result. LORO macro-F1 = **0.732** ± 0.118. Pooled OOF F1 = 0.758.

That is below the mean-pool logistic on the same tiles (0.761) and below the 128-D ViT with no contrastive term (0.756). Phoenix is the only metro with a clear gap in the 3D space (sep/within = 3.0). NYC and SF are one cloud.

```
.venv/bin/python -u scripts/train_week4_vit_100m_3d.py
.venv/bin/python -u scripts/build_week4_vit_100m_3d_notebook.py
```

1.1 Held-out scores

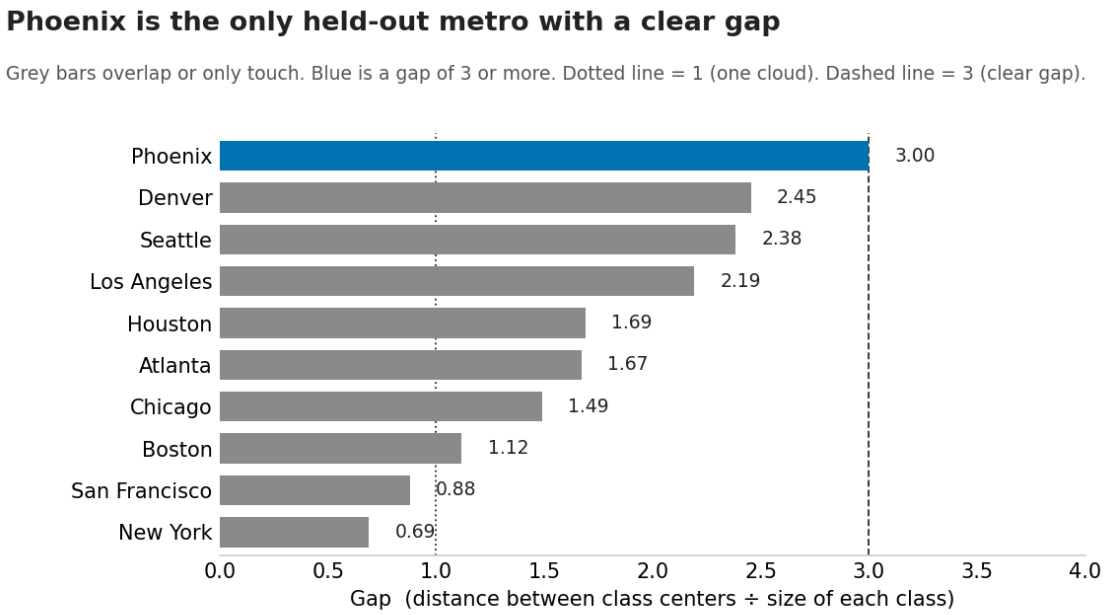
sep is the distance between class centroids. within is the mean distance to that centroid. sep/within above 3 is a visible gap. Near 1 the clouds overlap. Below 1 the centroids sit inside one blob.

Metro	F1	sep	within	sep/within
Phoenix_SW	0.888	12.27	4.09	3.00
Denver_W	0.867	9.53	3.88	2.45
Atlanta_SE	0.860	5.48	3.28	1.67
Seattle_NW	0.788	7.57	3.17	2.38
LA_W	0.786	5.86	2.67	2.19
Boston_NE	0.702	2.94	2.63	1.12
Houston_S	0.693	5.84	3.45	1.69
Chicago_MW	0.610	3.70	2.48	1.49
SF_W	0.573	2.60	2.97	0.88
NYC_NE	0.554	1.27	1.84	0.69

Chance for the contrastive term, at batch size 256, is about 5.54. A perfect cosine split floors near 4.84. The runs ended around 5.3, so the contrastive term never packed each class onto a point. sep grew, and within grew with it.

1.2 Where the classes separate

Gap is the distance between the two class centers divided by the average distance from a tile to its own center. Above 3 the groups are apart. Near 1 they are one cloud. The 3D figures below use the same order, largest gap first.

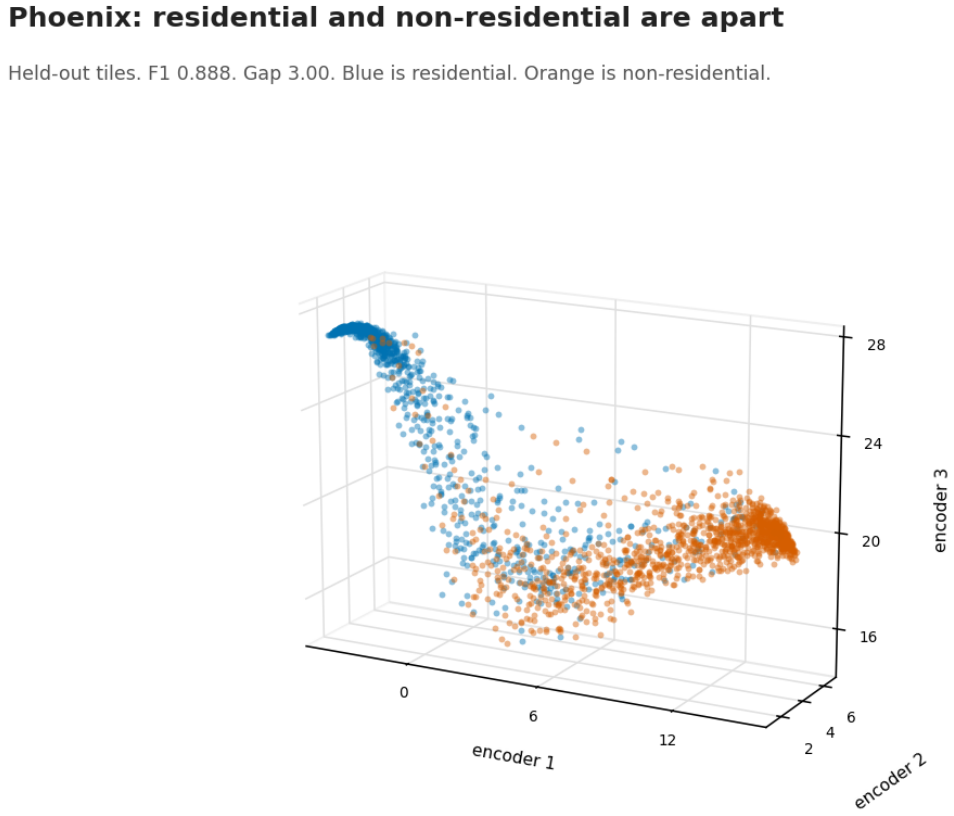




1.3 3D views

Each figure is that metro's own encoder output, largest gap first. The three axes are the three coordinates the model wrote.

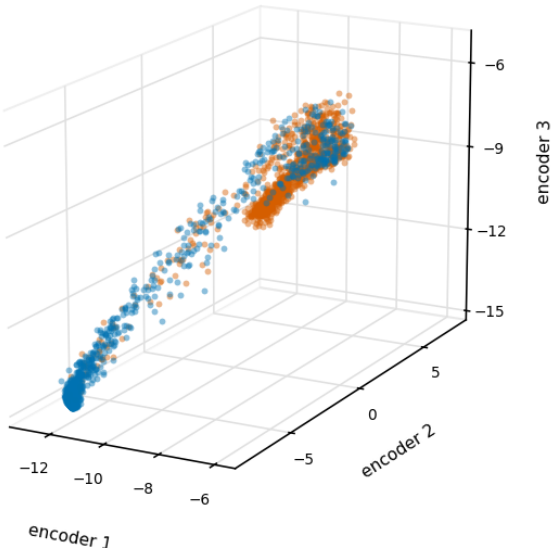
1.3.1 Phoenix



1.3.2 Denver

Denver: the two classes touch

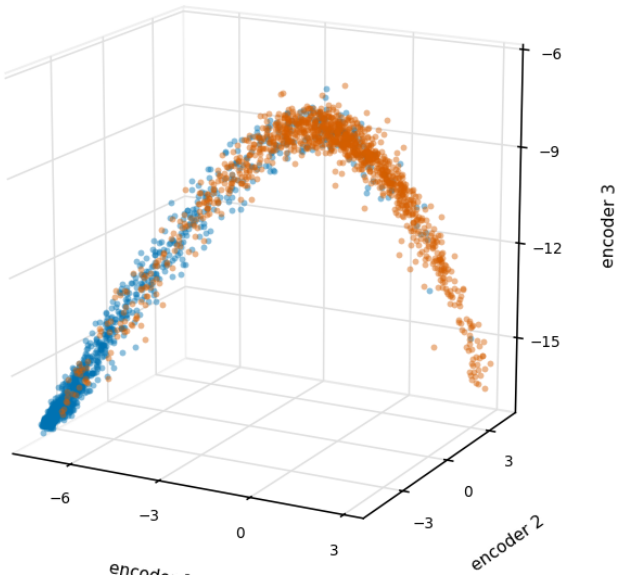
Held-out tiles. F1 0.867. Gap 2.45. Blue is residential. Orange is non-residential.



1.3.3 Seattle

Seattle: the two classes touch

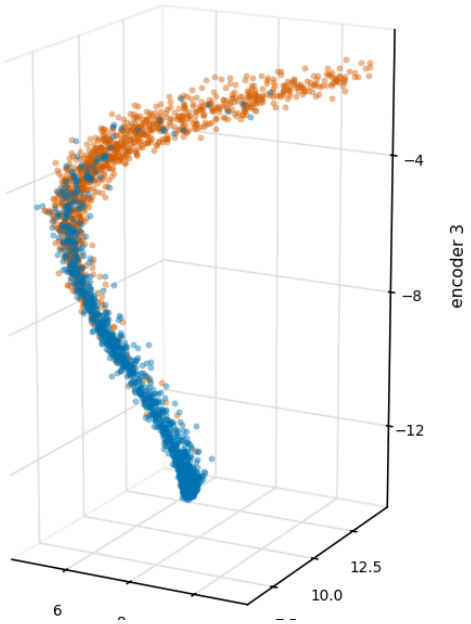
Held-out tiles. F1 0.788. Gap 2.38. Blue is residential. Orange is non-residential.



1.3.4 Los Angeles

Los Angeles: the two classes touch

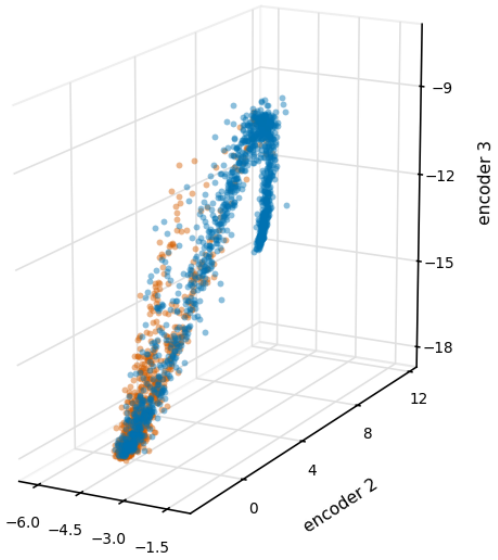
Held-out tiles. F1 0.786. Gap 2.19. Blue is residential. Orange is non-residential.



1.3.5 Houston

Houston: the two classes touch

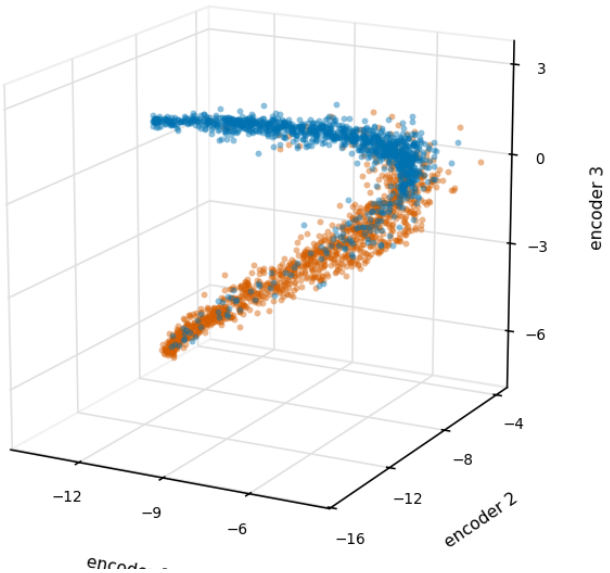
Held-out tiles. F1 0.693. Gap 1.69. Blue is residential. Orange is non-residential.



1.3.6 Atlanta

Atlanta: the two classes touch

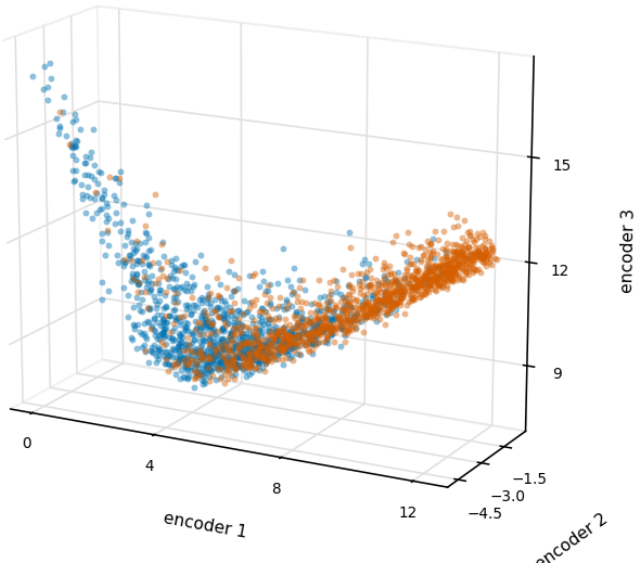
Held-out tiles. F1 0.860. Gap 1.67. Blue is residential. Orange is non-residential.



1.3.7 Chicago

Chicago: residential and non-residential are one cloud

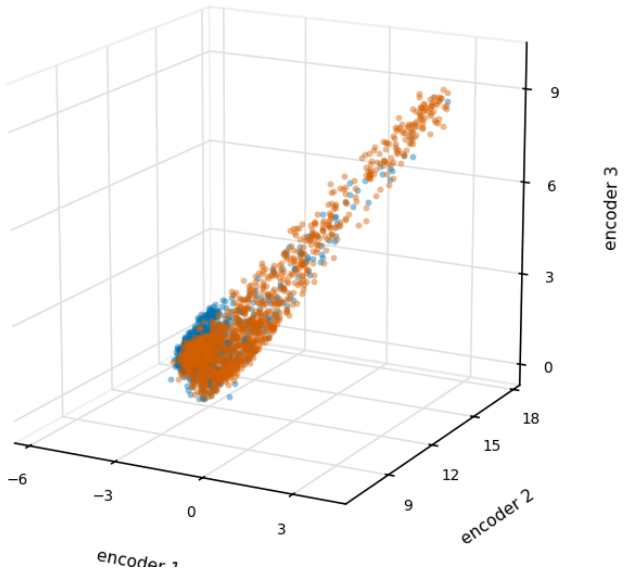
Held-out tiles. F1 0.610. Gap 1.49. Blue is residential. Orange is non-residential.



1.3.8 Boston

Boston: residential and non-residential are one cloud

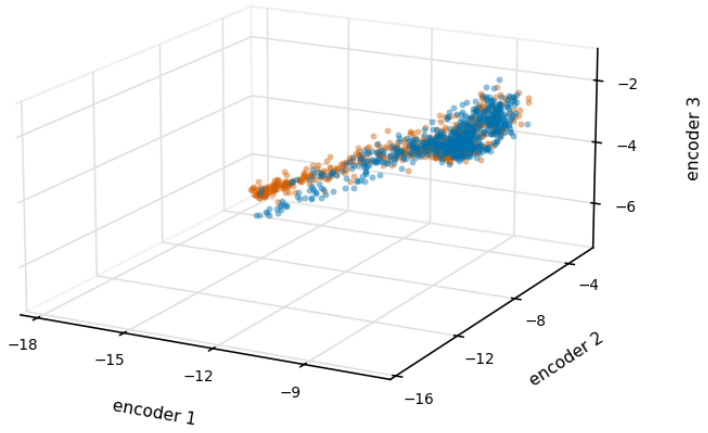
Held-out tiles. F1 0.702. Gap 1.12. Blue is residential. Orange is non-residential.



1.3.9 San Francisco

San Francisco: residential and non-residential are one cloud

Held-out tiles. F1 0.573. Gap 0.88. Blue is residential. Orange is non-residential.



1.3.10 New York



1.4 Code

The training driver is `scripts/train_week4_vit_100m_3d.py`. The pieces that define the 3D space are the encoder bottleneck and the contrastive loss. The next cell reloads the saved coordinates and redraws one metro.

[ ]:
```python
"""3D bottleneck ViT and supervised contrastive loss.

Matches scripts/train_week4_vit_100m_3d.py.
"""
import torch
import torch.nn as nn
import torch.nn.functional as F

EMB_DIM = 3
GRID = 10


def supervised_contrastive(z, y, tau: float = 0.1):
    """Pull same-class 3D vectors together, push the other class away."""
    u = F.normalize(z, dim=1)
    logits = u @ u.T / tau
```

```
    b = z.shape[0]
    eye = torch.eye(b, device=z.device, dtype=torch.bool)
    labels = y.view(-1).long()
    pos = labels[:, None].eq(labels[None, :]) & ~eye
    logits = logits.masked_fill(eye, -1e9)
    log_prob = logits - torch.logsumexp(logits, dim=1, keepdim=True)
    pos_count = pos.sum(dim=1)
    mean_log_prob = (log_prob * pos).sum(dim=1) / pos_count.clamp_min(1)
    valid = pos_count > 0
    if not bool(valid.any()):
        return z.new_zeros(())
    return -mean_log_prob[valid].mean()


class AEPatchViT3D(nn.Module):
    """Wide ViT over 10x10 PLS patches. Class token compressed to 3D."""

    def __init__(self, patch_dim, d_model=128, n_heads=4, n_layers=4, dropout=0.
↪1, emb_dim=EMB_DIM):
        super().__init__()
        n_patches = GRID * GRID
        self.patch_proj = nn.Linear(patch_dim, d_model)
        self.cls_token = nn.Parameter(torch.zeros(1, 1, d_model))
        self.pos_embed = nn.Parameter(torch.zeros(1, 1 + n_patches, d_model))
        layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=n_heads,
            dim_feedforward=int(d_model * 2),
            dropout=dropout,
            activation="gelu",
            batch_first=True,
            norm_first=True,
        )
        self.encoder = nn.TransformerEncoder(layer, num_layers=n_layers,␣
↪enable_nested_tensor=False)
        self.norm = nn.LayerNorm(d_model)
        self.to_3d = nn.Linear(d_model, emb_dim)
        self.head = nn.Linear(emb_dim, 1)

    def encode(self, x, mask):
        b = x.shape[0]
        tokens = self.patch_proj(x)
        cls = self.cls_token.expand(b, -1, -1)
        tokens = torch.cat([cls, tokens], dim=1) + self.pos_embed
        pad = torch.zeros(b, 1, device=x.device, dtype=torch.bool)
        key_pad = torch.cat([pad, mask < 0.5], dim=1)
        h = self.encoder(tokens, src_key_padding_mask=key_pad)
```

```
        return self.to_3d(self.norm(h[:, 0]))

    def forward(self, x, mask):
        z = self.encode(x, mask)
        return self.head(z).squeeze(-1), z


# Training step (inside each LORO fold, on that fold's GPU):
#   logits, z = model(x, mask)
#   loss = BCE(logits, y) + lambda * supervised_contrastive(z, y, tau=0.1)
```

[ ]:
```
"""Reload one metro and scatter its three encoder outputs.

Blue is residential, orange is non-residential.
Axes are the three encoder outputs.
"""
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

OUT = Path("../../outputs/week4_vit_100m_3d")
REGION = "Phoenix_SW"  # try NYC_NE for the overlapping cloud

oof = pd.read_parquet(OUT / "oof_predictions.parquet")
z = np.load(OUT / "oof_z3.npy")
m = oof["region"].to_numpy() == REGION
zz = z[m]
yy = oof.loc[m, "y_true"].to_numpy().astype(int)

fig = plt.figure(figsize=(7.4, 5.5))
ax = fig.add_subplot(111, projection="3d")
for lab, color in ((0, "#D55E00"), (1, "#0072B2")):
    sel = yy == lab
    ax.scatter(zz[sel, 0], zz[sel, 1], zz[sel, 2], s=9, c=color, alpha=0.45,␣
 ↪linewidths=0, depthshade=False)
spans = np.ptp(zz, axis=0)
spans = np.clip(np.maximum(spans, 1e-6) / spans.max(), 0.32, 1.0)
ax.set_box_aspect(tuple(spans))
ax.set_xlabel("encoder 1")
ax.set_ylabel("encoder 2")
ax.set_zlabel("encoder 3")
ax.view_init(elev=16, azim=-62)
ax.set_title(REGION.replace("_", " ") + "  ·  blue residential, orange␣
 ↪non-residential", loc="left")
plt.show()
```

1.5 Reload

Path	Contents
`outputs/week4_vit_100m_3d/checkpoints/fold_<metro>.pt`	Weights, PLS rotation, score mean/std, history
`outputs/week4_vit_100m_3d/oof_z3.npy`	Held-out 3D vectors, shape (52492, 3). Axes differ by metro.
`outputs/week4_vit_100m_3d/oof_predictions.parquet`	Labels and probabilities
`outputs/week4_vit_100m_3d/logs/<metro>.log`	Epoch bce, supcon, val F1, sep, within

Elapsed 537.7 s on 8 GPUs. Class: `scripts.train_week4_vit_100m_3d.AEPatchViT3D`.